Московский государственный технический университет
имени Н.Э. Баумана

Кафедра «Системы обработки информации и управления»

Ю.Е. Гапанюк

УЧЕБНЫЕ МАТЕРИАЛЫ СЕМИНАРА №6
РЕФАКТОРИНГ И ПРИНЦИПЫ SOLID

Учебные материалы по дисциплине
«Архитектура автоматизированных систем обработки информации и управления»
Профиль: «Архитектура программных систем»

Москва – 2026

Оглавление

1 ЦЕЛЬ ... 4

2 ЧТО ТАКОЕ «ЧИСТЫЙ КОД» ... 4

2.1 Мотивация ... 4

2.1.1 Эксперимент: два варианта одного метода ... 5

2.1.2 Правила чистого кода ... 6

3 ПРИНЦИПЫ DRY, KISS И YAGNI ... 8

3.1 DRY — Don't Repeat Yourself ... 8

3.2 KISS — Keep It Simple, Stupid ... 9

3.3 YAGNI — You Aren't Gonna Need It ... 10

3.4 Как соотносятся DRY, KISS и YAGNI ... 12

4 ПРИНЦИПЫ SOLID ... 12

4.1 S — Single Responsibility Principle ... 13

4.2 O — Open/Closed Principle ... 16

4.3 L — Liskov Substitution Principle ... 19

4.4 I — Interface Segregation Principle ... 21

4.5 D — Dependency Inversion Principle ... 23

4.6 Сводная таблица ... 26

5 РЕФАКТОРИНГ ... 26

5.1 Определение ... 26

5.2 Приёмы рефакторинга на одном примере ... 27

5.3 Сводка изменений ... 30

5.4 Инструменты Visual Studio ... 30

6 СИНТЕЗ ПАТТЕРНА «СТРАТЕГИЯ» ИЗ ПРИНЦИПОВ SOLID ... 31

6.1 Исходная задача ... 31

6.2 Шаг 1. Применяем OCP ... 32

6.3 Шаг 2. Проверяем SRP ... 33

6.4 Шаг 3. Проверяем DIP ... 33

6.5 Шаг 4. Проверяем LSP ... 34

6.6 Шаг 5. Проверяем ISP ... 34

6.7 Что получилось ... 34

6.8 Использование в точке сборки ... 34

6.9 Выводы по разделу ... 35

7 ИСПОЛЬЗОВАНИЕ ИИ ДЛЯ РЕФАКТОРИНГА ... 36

7.1 Практический эксперимент ... 36

7.2 Анализ результата ... 36

7.3 Рекомендуемый порядок работы 37
7.4 Главный тезис 37

8 ПРАКТИЧЕСКОЕ ЗАДАНИЕ 37

8.1 Условие 37
8.2 Требования к выполнению 39
8.3 Ожидаемые нарушения 39

9 ВЫВОДЫ 40

1 Цель

Освоить понятие чистого кода и основные принципы, которыми руководствуются при проектировании поддерживаемых программных систем. Познакомиться с принципами SOLID на примерах из предметной области курса. Освоить основные приемы рефакторинга и научиться видеть в реальных задачах проектирования проявление изученных принципов.

2 Что такое «чистый код»

2.1 Мотивация

На предыдущих занятиях мы научились использовать основные возможности языка C#: классы, наследование, полиморфизм, интерфейсы, познакомились с принципами внедрения зависимостей. Этого инструментария достаточно для того, чтобы написать программу, которая решает поставленную задачу. Но работающий код – еще не обязательно хороший код.

Роберт Мартин в книге «Чистый код» формулирует наблюдение, ставшее практически общепризнанным: соотношение времени чтения кода к времени его написания составляет более чем 10:1. Разработчик читает старый код — свой и чужой — постоянно: при ревью, при исправлении ошибок, при добавлении новой функциональности, при попытке понять, что здесь вообще происходит. Следовательно, облегчить чтение кода — значит облегчить всю последующую работу над ним.

Код, который тяжело читать, принято называть «грязным». Он накапливает технический долг: каждое следующее изменение в нём становится дороже предыдущего, до тех пор, пока стоимость поддержки не превысит стоимость полного переписывания.

2.1.1 Эксперимент: два варианта одного метода

Рассмотрим метод, рассчитывающий годовую премию сотрудников отдела. Ниже показаны два варианта одного и того же алгоритма. Оба варианта компилируются и дают одинаковый результат на одинаковых входных данных.

«Грязный» вариант:

```
class A
{
    public double Calc(List<int[]> d, int x, int y)
    {
        double r = 0;
        for (int i = 0; i < d.Count; i++)
        {
            if (d[i][0] == x)
                if (d[i][1] > 0)
                    if (d[i][2] == y)
                        if (d[i][3] > 5)
                            if (d[i][4] == 1)
                                r += d[i][1] * 0.15;
                            else
                                r += d[i][1] * 0.12;
                        else
                            r += d[i][1] * 0.07;
        }
        return r;
    }
}
```

Попробуем ответить на следующие вопросы:

- что делает этот метод;
- что хранится в d[i][3] и почему это значение сравнивается с числом 5;
- откуда взяты коэффициенты 0,15, 0,12 и 0,07;
- что понадобится изменить, если коэффициент премии для одной из категорий сотрудников изменится?

«Чистый» вариант:

```
class BonusCalculator
{
    private const double SeniorFullTimeBonusRate = 0.15;
    private const double JuniorFullTimeBonusRate = 0.12;
    private const double PartTimeBonusRate       = 0.07;
    private const int    SeniorExperienceYears   = 5;
```

```
    public double CalculateDepartmentAnnualBonus(
        List<Employee> employees,
        int departmentId,
        int bonusYear)
    {
        return employees
            .Where(e => e.DepartmentId == departmentId && e.BonusYear==bonusYear)
            .Sum(CalculateEmployeeBonus);
    }

    private double CalculateEmployeeBonus(Employee employee)
    {
        if (employee.Salary <= 0)
            return 0;

        if (!employee.IsFullTime)
            return employee.Salary * PartTimeBonusRate;

        return employee.YearsOfExperience > SeniorExperienceYears
            ? employee.Salary * SeniorFullTimeBonusRate
            : employee.Salary * JuniorFullTimeBonusRate;
    }
}
```

Теперь те же вопросы получают ответ немедленно — код объясняет себя сам. При этом алгоритм не изменился: изменились только имена, структура и способ подачи информации.

2.1.2 Правила чистого кода

Сформулируем основные правила, которые будут применяться далее на протяжении всего семинара.

Именование. В языке C# приняты соглашения, которыми мы уже пользовались: классы, методы и свойства — PascalCase; локальные переменные и параметры — camelCase; приватные поля — _camelCase. Имя должно отражать смысл сущности: не d, а employees; не Calc, а CalculateAnnualBonus.

Длина метода. Метод длиной более 20–30 строк почти всегда делает несколько дел одновременно, и его следует разбить на несколько более мелких методов. Короткий метод легче понять, легче протестировать и легче переиспользовать.

Уровень вложенности. Более двух-трёх уровней вложенности if и циклов делают код тяжёлым для восприятия. Типичный приём устранения глубокой вложенности — «ранний return»: проверка всех нештатных ситуаций в начале метода с немедленным выходом. Сравним:

```
// Глубокое вложение: смысл «прячется» вглубь
public string GetEmployeeStatus(Employee employee)
{
    string status = "";
    if (employee != null)
    {
        if (employee.IsActive)
        {
            if (employee.YearsOfExperience > 5)
                status = "Старший специалист";
            else
                status = "Специалист";
        }
        else
        {
            status = "Уволен";
        }
    }
    return status;
}

// Ранний return: читается линейно сверху вниз
public string GetEmployeeStatus(Employee employee)
{
    if (employee == null)    return string.Empty;
    if (!employee.IsActive)  return "Уволен";

    return employee.YearsOfExperience > 5
        ? "Старший специалист"
        : "Специалист";
}
```

Правильно написанный код не нуждается в комментариях, объясняющих, что он делает — это видно из самого кода. Комментарий становится уместен, когда он объясняет, почему код написан именно так. Например:

```
// Плохо: комментарий дублирует код, ничего не добавляя
salary = salary * 0.87;  // умножаем зарплату на 0.87

// Хорошо: комментарий поясняет нетривиальное решение
// Коэффициент 0,87 = 1 – НДФЛ 13 %; закреплён приказом № 47 от 01.01.2024
decimal netSalary = grossSalary * NetSalaryCoefficient;
```

Комментарии, объясняющие очевидное, не только бесполезны, но и вредны: они устаревают раньше, чем сам код, и начинают вводить в заблуждение.

3 Принципы DRY, KISS и YAGNI

Три принципа, которые будут рассмотрены в этом разделе, близки по духу и часто упоминаются вместе. Их объединяет общая идея: бороться с ненужной сложностью.

3.1 DRY — Don’t Repeat Yourself

Принцип DRY (Эндрю Хант и Дэвид Томас, «Программист-прагматик») формулируется так: каждый фрагмент знания должен иметь единственное, непротиворечивое представление в системе.

Рассмотрим типичное нарушение: два метода проверяют корректность данных книги одинаковым образом.

```
class LibraryServiceBad
{
    public void AddBook(string title, string author, int year)
    {
        if (string.IsNullOrWhiteSpace(title))
            throw new ArgumentException("Название не может быть пустым");
        if (string.IsNullOrWhiteSpace(author))
            throw new ArgumentException("Автор не может быть пустым");
        if (year < 1000 || year > DateTime.Now.Year)
            throw new ArgumentException("Некорректный год издания");
        // ... логика добавления ...
    }

    public void UpdateBook(int id, string title, string author, int year)
    {
        // Точная копия блока валидации из AddBook
        if (string.IsNullOrWhiteSpace(title))
            throw new ArgumentException("Название не может быть пустым");
        if (string.IsNullOrWhiteSpace(author))
            throw new ArgumentException("Автор не может быть пустым");
        if (year < 1000 || year > DateTime.Now.Year)
            throw new ArgumentException("Некорректный год издания");
        // ... логика обновления ...
    }
}
```

Правило «минимальный допустимый год издания — 1000» — это знание. Оно представлено в коде дважды. Если знание изменится (например, минимальным станет 1450-й — год изобретения книгопечатания), мы обязаны исправить обе копии. Если таких копий окажется двадцать, а одну из них кто-то забудет исправить — в системе появится противоречие, которое обнаружится не сразу.

Исправление состоит в выделении единственного представления правила:

```
class LibraryServiceGood
{
    public void AddBook(string title, string author, int year)
    {
        ValidateBookData(title, author, year);
        // ... логика добавления ...
    }

    public void UpdateBook(int id, string title, string author, int year)
    {
        ValidateBookData(title, author, year);
        // ... логика обновления ...
    }

    private void ValidateBookData(string title, string author, int year)
    {
        if (string.IsNullOrWhiteSpace(title))
            throw new ArgumentException("Название не может быть пустым");
        if (string.IsNullOrWhiteSpace(author))
            throw new ArgumentException("Автор не может быть пустым");
        if (year < 1000 || year > DateTime.Now.Year)
            throw new ArgumentException("Некорректный год издания");
    }
}
```

3.2 KISS — Keep It Simple, Stupid

Принцип KISS предписывает не усложнять решение сверх того, что требуется задачей. Рассмотрим задачу: рассчитать штраф за просроченную книгу, равный пяти рублям за день просрочки.

Первый разработчик пишет:

```
decimal CalculateFine(int daysOverdue) => daysOverdue > 0 ? daysOverdue *
5m : 0m;
```

Второй разработчик, только что прочитавший книгу о паттернах проектирования, пишет:

```
interface IFineStrategy { decimal Calculate(int daysOverdue); }

abstract class FineStrategyBase : IFineStrategy
{
    public abstract decimal Calculate(int daysOverdue);
}

class NoFineStrategy : FineStrategyBase
{
    public override decimal Calculate(int daysOverdue) => 0m;
}

class StandardFineStrategy : FineStrategyBase
{
    public override decimal Calculate(int daysOverdue) => daysOverdue * 5m
;
}

class FineStrategyFactory
{
    public static IFineStrategy Create(bool isOverdue)
        => isOverdue ? new StandardFineStrategy() : new NoFineStrategy();
}
```

Второе решение делает то же самое, что и первое, — но вводит пять классов и одну фабрику там, где достаточно одного выражения. Такое явление называется оверинжинирингом (пере-проектированием, чрезмерным проектированием). Его цена — чужое время на чтение кода, дополнительные места для ошибок и иллюзия архитектурной сложности там, где её нет.

KISS не запрещает сложные решения — он запрещает сложные решения простых задач. Если у библиотеки действительно появятся разные категории читателей с разными правилами начисления штрафа, стратегии станут оправданы.

3.3 YAGNI — You Aren’t Gonna Need It

Принцип YAGNI (Кент Бек, Extreme Programming) формулируется кратко: не реализуй функциональность до того момента, когда она действительно потребуется.

YAGNI — это обещание, которое программист даёт самому себе: «Я не буду писать код на случай, если когда-нибудь он может понадобиться. Я напишу его, когда он действительно понадобится».

Рассмотрим типичное нарушение. Заказчик попросил сохранять книгу в файл. Разработчик, стараясь «сделать всё по уму», пишет:

```
class BookStorage
{
    // Реализовано — этого и просили
    public void SaveToFile(Book book, string path) { /* работающий код */
}

    // «Вдруг понадобится» — написаны заглушки
    public void SaveToXml(Book book, string path)        { /* TODO */ }
    public void SaveToJson(Book book, string path)       { /* TODO */ }
    public void SaveToDatabase(Book book, string cs)     { /* TODO */ }
    public void SaveToCloud(Book book, string bucket)    { /* TODO */ }

    // «Вдруг понадобится кэширование»
    private Dictionary<int, Book> _cache = new();

    // «Вдруг понадобится асинхронная версия»
    public async Task SaveToFileAsync(Book b, string p) { /* TODO */ }
}
```

Все эти «на всякий случай» — потерянное время, лишние места для ошибок, лишний код, который коллеги должны читать и поддерживать, не получая взамен никакой пользы. Практика показывает, что большая часть такого кода либо никогда не понадобится, либо понадобится, но в другой форме, и всё равно будет переписана.

Соблюдение YAGNI означает, что пока требования формулируются как «сохранять в файл», мы реализуем только это:

```
class BookStorage
{
    public void SaveToFile(Book book, string path)
    {
        File.WriteAllText(path, $"{book.Title}|{book.Author}");
    }
}
```

Когда появится требование «сохранять в JSON» — добавим метод. Не раньше.

3.4 Как соотносятся DRY, KISS и YAGNI

Принципы близки, но разграничиваются так:

Принцип	Основная идея	С чем борется
DRY	Каждое знание — в одном месте	С дублированием уже существующего
KISS	Не усложняй имеющееся	С лишней сложностью в решении
YAGNI	Не пиши того, что не нужно	С избыточной функциональностью «впрок»

Эти принципы иногда входят в противоречие. Устранение дублирования по DRY может породить абстрактный класс, нарушающий KISS. YAGNI подсказывает, что два похожих фрагмента пока не обязательно сводить к общему: возможно, они лишь кажутся одинаковыми и будут расходиться в разные стороны. Существует эмпирическое «правило трёх»: устранять дублирование имеет смысл при появлении третьей копии — до этого объединение может оказаться преждевременным.

Опытный разработчик не следует этим принципам буквально — он удерживает их в поле зрения и в каждом конкретном случае выбирает разумный компромисс.

4 Принципы SOLID

Акроним SOLID, предложенный Робертом Мартином, обозначает пять принципов объектно-ориентированного проектирования. Каждый принцип отвечает на отдельный вопрос о хорошо спроектированном классе:

Принцип	Ключевой вопрос
S — Single Responsibility Principle	У класса одна причина для изменения?
O — Open/Closed Principle	Расширение возможно без изменения существующего кода?
L — Liskov Substitution Principle	Наследник корректно замещает базовый класс?

Принцип	Ключевой вопрос
I — Interface Segregation Principle	Клиент не зависит от методов, которые ему не нужны?
D — Dependency Inversion Principle	Зависимости направлены к абстракциям?

Каждый принцип разбирается ниже по общей схеме: формулировка, два примера нарушения из разных предметных областей, исправление и связь с уже изученными конструкциями C#. Два примера нужны для того, чтобы увидеть принцип с разных сторон: один и тот же принцип может проявляться по-разному, и умение распознать его в непривычной обстановке — признак того, что материал действительно усвоен.

4.1 S — Single Responsibility Principle

Формулировка. У класса должна быть одна и только одна причина для изменения.

Пример 1. Класс сотрудника. Рассмотрим класс Employee, который взял на себя четыре не связанных между собой задачи:

```
class Employee
{
    // Ответственность 1: хранение данных
    public int     Id                { get; set; }
    public string  Name              { get; set; } = string.Empty;
    public string  Department        { get; set; } = string.Empty;
    public decimal Salary            { get; set; }
    public int     YearsOfExperience { get; set; }

    // Ответственность 2: валидация
    public bool IsValid()
        => !string.IsNullOrWhiteSpace(Name) && Salary > 0;

    // Ответственность 3: сохранение в базу данных
    public void SaveToDatabase(string connectionString)
    {
        // using var conn = new SqliteConnection(connectionString);
        // conn.Execute("INSERT INTO Employees ...", this);
    }

    // Ответственность 4: формирование отчёта
    public string GeneratePayslip()
    {
        decimal tax    = Salary * 0.13m;
        decimal netPay = Salary - tax;
        return $"Расчётный листок\nСотрудник: {Name}\n" +
```

```
                $"Начислено: {Salary:C}\nНДФЛ: {tax:C}\nК выплате: {netPay:
C}";
    }
}
```

Этот класс будет изменяться по четырём разным причинам: при изменении структуры данных сотрудника, при изменении бизнес-правил валидации, при смене СУБД, при изменении формата расчётного листка. Каждая из этих причин принадлежит своей области: первая — отделу кадров, вторая — бизнес-аналитикам, третья — инженерам по инфраструктуре, четвёртая — бухгалтерии. Класс, затрагиваемый столькими разными интересами, становится точкой постоянных конфликтов.

Пример 2. Класс заказа в интернет-магазине. Проследим ту же проблему в другой предметной области:

```
class Order
{
    public int Id { get; set; }
    public List<OrderItem> Items { get; set; } = new();
    public decimal TotalAmount { get; set; }

    // Ответственность 1: расчёт налогов
    public decimal CalculateTax()
    {
        // ... сложная логика расчёта НДС в зависимости от товара и регион
а ...
        return TotalAmount * 0.20m;
    }

    // Ответственность 2: отправка электронной почты
    public void SendConfirmationEmail(string customerEmail)
    {
        // SmtpClient client = new SmtpClient("smtp.company.com");
        // client.Send(new MailMessage(...));
    }

    // Ответственность 3: печать квитанции
    public void PrintReceipt()
    {
        // PrinterSettings settings = new PrinterSettings();
        // ...
    }

    // Ответственность 4: сохранение в базу
    public void Save() { /* запись в БД */ }
}
```

Этот класс столкнётся с теми же симптомами: при смене ставки НДС — правим Order; при переходе с SMTP на сервис уведомлений — правим Order; при смене формата квитанции — правим Order. При этом у изменений нет ничего общего, и их объединение в одном классе — произвол.

Исправление в обоих случаях одно и то же: каждая ответственность выносится в отдельный класс. Покажем, как это выглядит для Employee:

```
/// <summary>
/// Хранение данных сотрудника.
/// Причина изменения: изменилась структура данных.
/// </summary>
class Employee
{
    public int     Id                { get; set; }
    public string  Name              { get; set; } = string.Empty;
    public string  Department        { get; set; } = string.Empty;
    public decimal Salary            { get; set; }
    public int     YearsOfExperience { get; set; }
}

/// <summary>
/// Валидация бизнес-правил.
/// Причина изменения: изменились правила валидации.
/// </summary>
class EmployeeValidator
{
    public bool IsValid(Employee employee)
    {
        if (string.IsNullOrWhiteSpace(employee.Name)) return false;
        if (employee.Salary <= 0)                     return false;
        return true;
    }
}

/// <summary>
/// Доступ к базе данных.
/// Причина изменения: сменилась СУБД или способ хранения.
/// </summary>
class EmployeeRepository
{
    private readonly string _connectionString;
    public EmployeeRepository(string connectionString)
        => _connectionString = connectionString;

    public void      Save(Employee employee) { /* INSERT/UPDATE */ }
    public Employee? GetById(int id)         { /* SELECT */ return null; }
}

/// <summary>
```

```
/// Формирование расчётного листка.
/// Причина изменения: изменился формат отчёта.
/// </summary>
class PayslipGenerator
{
    private const decimal IncomeTaxRate = 0.13m;

    public string Generate(Employee employee)
    {
        decimal tax    = employee.Salary * IncomeTaxRate;
        decimal netPay = employee.Salary - tax;
        return $"Расчётный листок\n" +
               $"Сотрудник: {employee.Name}\n" +
               $"Отдел: {employee.Department}\n" +
               $"Начислено: {employee.Salary:C}\n" +
               $"НДФЛ (13 %): {tax:C}\n" +
               $"К выплате: {netPay:C}";
    }
}
```

Теперь каждый класс имеет одну причину для изменения, и автор изменений в одной области не затрагивает работу остальных.

Уместно обратить внимание на разницу между техническим и архитектурным разделением класса. Ключевое слово partial позволяет разделить один класс на несколько файлов — это организационный приём. Принцип SRP разделяет не файлы, а ответственности — это архитектурный приём. Класс можно разнести на пять файлов через partial и всё равно нарушить SRP.

4.2 O — Open/Closed Principle

Формулировка. Программная сущность должна быть открыта для расширения, но закрыта для модификации.

Иными словами, при добавлении нового поведения не должно возникать необходимости изменять уже написанный и работающий код.

Пример 1. Отчёт по элементам библиотеки. Рассмотрим класс, выводящий информацию об элементах библиотечной системы:

```
class LibraryItemReporter
{
    public void PrintInfo(object item)
    {
        if (item is Book book)
```

```
            Console.WriteLine($"[КНИГА] {book.Title} — {book.Author}");
        else if (item is Magazine magazine)
            Console.WriteLine($"[ЖУРНАЛ] {magazine.Title}, №{magazine.Issu
eNumber}");
        else if (item is Dvd dvd)
            Console.WriteLine($"[DVD] {dvd.Title}, {dvd.DurationMinutes} м
ин.");
        // Добавляем аудиокнигу — снова редактируем этот метод
    }
}
```

Каждый новый тип элемента требует правки этого метода. Это нарушает OCP: класс не закрыт для модификации. Более того, каждая правка — это риск: проверенный и работающий код приходится затрагивать ради функциональности, к нему прямо не относящейся. Исправление — использование абстракции с виртуальным методом:

```
abstract class LibraryItem
{
    public string Title { get; set; } = string.Empty;
    public abstract string GetDisplayInfo();
}

class Book : LibraryItem
{
    public string Author { get; set; } = string.Empty;
    public int    Year   { get; set; }
    public override string GetDisplayInfo()
        => $"[КНИГА] {Title} — {Author} ({Year})";
}

class Magazine : LibraryItem
{
    public int IssueNumber { get; set; }
    public override string GetDisplayInfo()
        => $"[ЖУРНАЛ] {Title}, №{IssueNumber}";
}

// Новый тип добавляется без единой правки существующих классов
class Dvd : LibraryItem
{
    public int DurationMinutes { get; set; }
    public override string GetDisplayInfo()
        => $"[DVD] {Title}, {DurationMinutes} мин.";
}

class LibraryItemReporter
{
    public void PrintInfo(LibraryItem item)
        => Console.WriteLine(item.GetDisplayInfo());
}
```

Пример 2. Расчёт скидки. Тот же принцип можно увидеть в логике скидок для разных категорий покупателей:

```
// Нарушение OCP
class PriceCalculator
{
    public decimal GetFinalPrice(decimal basePrice, string customerType)
    {
        if (customerType == "Regular")
            return basePrice;
        else if (customerType == "Student")
            return basePrice * 0.90m;
        else if (customerType == "Senior")
            return basePrice * 0.85m;
        else if (customerType == "Staff")
            return basePrice * 0.70m;
        // При добавлении новой категории — снова правим этот метод
        else
            return basePrice;
    }
}
```

Проблема та же, что и в первом примере, — она просто проявляется через другой механизм (строковый идентификатор вместо is-проверки). Исправление:

```
abstract class Customer
{
    public abstract decimal GetDiscountRate();

    public decimal GetFinalPrice(decimal basePrice)
        => basePrice * (1 - GetDiscountRate());
}

class RegularCustomer : Customer  { public override decimal GetDiscountRat
e() => 0.00m; }
class StudentCustomer : Customer  { public override decimal GetDiscountRat
e() => 0.10m; }
class SeniorCustomer  : Customer  { public override decimal GetDiscountRat
e() => 0.15m; }
class StaffCustomer   : Customer  { public override decimal GetDiscountRat
e() => 0.30m; }
```

Общее в двух примерах: ветвление по типу заменяется полиморфизмом, и новое поведение добавляется через наследование, а не через правку существующего кода.

Замечание о YAGNI. OCP не означает, что в каждом классе заранее следует предусмотреть «точки расширения». Абстракцию вводят тогда,

когда появляется реальная потребность — обычно при добавлении второй реализации. До этого преждевременно проектированная абстракция рискует оказаться ненужной или неверной.

4.3 L — Liskov Substitution Principle

Формулировка. Объекты подклассов должны быть замещаемы объектами базового класса без изменения корректности программы.

Таким образом, код, написанный в расчёте на базовый класс, должен продолжать работать корректно, если вместо него передать объект подкласса.

Пример 1. Прямоугольник и квадрат. Классический пример нарушения LSP. С математической точки зрения квадрат — частный случай прямоугольника, и соблазнительно отразить это отношением наследования:

```
class Rectangle
{
    public virtual int Width  { get; set; }
    public virtual int Height { get; set; }
    public int Area() => Width * Height;
}

class Square : Rectangle
{
    // Квадрат принуждает обе стороны быть равными
    public override int Width
    {
        get => base.Width;
        set { base.Width = value; base.Height = value; }
    }
    public override int Height
    {
        get => base.Height;
        set { base.Width = value; base.Height = value; }
    }
}
```

Теперь представим код, который работает с Rectangle и рассчитывает на его поведение:

```
void StretchHorizontally(Rectangle rect)
{
    rect.Width  = 10;
```

```
    rect.Height = 3;
    Console.WriteLine($"Площадь: {rect.Area()}");
}
```

Для Rectangle этот метод выведет 30, что соответствует ожиданиям. Для Square он выведет 9: установка Height = 3 изменила и Width. Код, корректный для базового класса, перестал быть корректным для подкласса. LSP нарушен.

Причина нарушения — в коде, а не в математике: наследование в ООП означает не «является», а «обязуется вести себя как». Квадрат, принуждая стороны быть равными, это обязательство нарушает.

Пример 2. Птицы. Второй классический пример. Предположим, мы проектируем иерархию птиц:

```
class Bird
{
    public virtual void Fly()
        => Console.WriteLine("Лечу");
}

class Sparrow : Bird { /* наследует Fly без изменений */ }

class Penguin : Bird
{
    public override void Fly()
        => throw new NotSupportedException("Пингвины не летают");
}
```

Код, работающий с любыми птицами, рассчитывает на то, что метод Fly не падает с исключением:

```
void ReleaseToSky(List<Bird> birds)
{
    foreach (var bird in birds)
        bird.Fly(); // для пингвина — NotSupportedException
}
```

Исправление в обоих случаях — не наследовать то, что нельзя наследовать. Пересмотрим иерархию, отделив летающих птиц от остальных:

```
abstract class Bird { /* общие свойства: имя, вес, ... */ }

abstract class FlyingBird : Bird
{
    public abstract void Fly();
```

```
}

class Sparrow : FlyingBird
{
    public override void Fly() => Console.WriteLine("Лечу");
}

class Penguin : Bird { /* плавает, но не летает */ }
```

Для прямоугольника исправление аналогично — ввести общего предка, не навязывающего лишних обязательств:

```
abstract class Shape
{
    public abstract int Area();
}

class Rectangle : Shape
{
    public int Width  { get; set; }
    public int Height { get; set; }
    public override int Area() => Width * Height;
}

class Square : Shape
{
    public int Side { get; set; }
    public override int Area() => Side * Side;
}
```

Практический признак нарушения LSP. Появление в подклассе NotImplementedException, NotSupportedException, а также «тихих» побочных эффектов, не предусмотренных контрактом базового класса, — верный признак того, что LSP нарушен и иерархию следует пересмотреть.

4.4 I — Interface Segregation Principle

Формулировка. Класс, реализующий интерфейс, не должен зависеть от методов, которые он не использует.

Пример 1. Интерфейс библиотечного элемента. Рассмотрим интерфейс, описывающий «всё, что может библиотечный элемент»:

```
interface ILibraryItem
{
    string GetInfo();
    void   Save();
    void   Print();
    byte[] ExportToPdf();
```

```
    bool   Search(string query);
}
```

Такой интерфейс называют «толстым». Он обязывает каждую реализацию уметь всё — даже то, что по смыслу ей не свойственно:

```
class ArchiveDocument : ILibraryItem
{
    public string GetInfo()            => "Архивный документ: секретно";
    public void   Save()               { /* корректная реализация */ }
    public bool   Search(string query) => false;

    // Вынужденные заглушки, нарушающие к тому же и LSP
    public void   Print()       => throw new NotSupportedException();
    public byte[] ExportToPdf() => throw new NotSupportedException();
}
```

Клиент, работающий через ILibraryItem, вынужден допускать, что некоторые методы могут не работать. Это превращает типовую систему в подобие неявного набора соглашений.

Исправление — разделить «толстый» интерфейс на несколько узких:

```
interface ISearchable  { bool   Search(string query); }
interface IPrintable   { void   Print();              }
interface IExportable  { byte[] ExportToPdf();        }
interface IPersistable { void   Save();               }

class Book : ISearchable, IPrintable, IExportable, IPersistable
{
    public string Title  { get; set; } = string.Empty;
    public string Author { get; set; } = string.Empty;

    public bool   Search(string query) => Title.Contains(query) || Author.
Contains(query);
    public void   Print()              => Console.WriteLine($"{Title} — {A
uthor}");
    public byte[] ExportToPdf()        => Array.Empty<byte>();
    public void   Save()               { /* сохранение */ }
}

class ArchiveDocument : ISearchable, IPersistable
{
    public string Code { get; set; } = string.Empty;
    public bool Search(string query) => Code.Contains(query);
    public void Save()               { /* сохранение */ }
}

// Метод принимает ровно тот интерфейс, который ему нужен
void ExportAll(IEnumerable<IExportable> items)
{
    foreach (var item in items)
```

```
        _ = item.ExportToPdf();
}
```

Пример 2. Интерфейс работника. Тот же принцип в другой предметной области. Пусть мы проектируем систему управления персоналом, где работник умеет три вещи: трудиться, обедать и спать:

```
interface IWorker
{
    void Work();
    void Eat();
    void Sleep();
}

class HumanWorker : IWorker
{
    public void Work()  { /* работает */ }
    public void Eat()   { /* обедает */ }
    public void Sleep() { /* спит */ }
}

// Робот тоже работает — но не ест и не спит
class RobotWorker : IWorker
{
    public void Work()  { /* работает */ }
    public void Eat()   => throw new NotSupportedException();
    public void Sleep() => throw new NotSupportedException();
}
```

Проблема та же. Исправление — разделить интерфейс по естественным границам обязанностей:

```
interface IWorkable   { void Work();  }
interface IFeedable   { void Eat();   }
interface ISleepable  { void Sleep(); }

class HumanWorker : IWorkable, IFeedable, ISleepable { /* ... */ }
class RobotWorker : IWorkable                        { /* ... */ }
```

4.5 D — Dependency Inversion Principle

Формулировка. 1. Модули верхнего уровня не должны зависеть от модулей нижнего уровня — и те, и другие должны зависеть от абстракций. 2. Абстракции не должны зависеть от деталей — детали должны зависеть от абстракций.

Пример 1. Библиотека с хранилищем и логгером. Рассмотрим класс Library, жёстко связанный с конкретными реализациями:

```
class Library
{
    // Жёсткая привязка к SQLite
    private SqliteBookRepository _repository = new SqliteBookRepository("c
atalog.db");

    // Жёсткая привязка к консольному логгеру
    private ConsoleLogger _logger = new ConsoleLogger();

    public void AddItem(LibraryItem item)
    {
        _repository.Save(item);
        _logger.Log($"Добавлен: {item.GetDisplayInfo()}");
    }
}
```

Класс Library относится к верхнему уровню (бизнес-логика), а SqliteBookRepository и ConsoleLogger — к нижнему (инфраструктура). Сейчас бизнес-логика зависит от инфраструктуры: при переходе с SQLite на PostgreSQL или с консоли на файл — придётся править Library.

Исправление — перевернуть направление зависимости через интерфейсы:

```
interface ILibraryRepository
{
    void                      Save(LibraryItem item);
    LibraryItem?              GetById(int id);
    IEnumerable<LibraryItem> GetAll();
}

class Library
{
    private readonly ILibraryRepository _repository;
    private readonly ILogger            _logger;

    public Library(ILibraryRepository repository, ILogger logger)
    {
        _repository = repository;
        _logger     = logger;
    }

    public void AddItem(LibraryItem item)
    {
        _repository.Save(item);
        _logger.Log($"Добавлен: {item.GetDisplayInfo()}");
    }
}

// Детали — в реализациях
class SqliteLibraryRepository   : ILibraryRepository { /* продакшен */ }
class InMemoryLibraryRepository : ILibraryRepository { /* тесты */ }
```

Теперь и Library, и SqliteLibraryRepository зависят от ILibraryRepository. Стрелка зависимости повернулась «снизу вверх» — отсюда и слово «инверсия» в названии принципа.

Пример 2. Сервис уведомлений. Та же идея в другой предметной области:

```
// Нарушение DIP
class OrderProcessingService
{
    // Бизнес-логика жёстко привязана к конкретному каналу уведомлений
    private EmailSender _emailSender = new EmailSender("smtp.company.com")
;

    public void PlaceOrder(Order order)
    {
        // ... обработка заказа ...
        _emailSender.SendEmail(order.CustomerEmail, "Заказ принят", "...")
;
    }
}
```

Захотели добавить уведомления по SMS — правим OrderProcessingService. Захотели push-уведомления — правим снова. Для каждого теста бизнес-логики приходится реально отправлять письмо.

Исправление:

```
interface INotificationService
{
    void Notify(string recipient, string subject, string body);
}

class OrderProcessingService
{
    private readonly INotificationService _notifications;

    public OrderProcessingService(INotificationService notifications)
    {
        _notifications = notifications;
    }

    public void PlaceOrder(Order order)
    {
        // ... обработка заказа ...
        _notifications.Notify(order.CustomerEmail, "Заказ принят", "...");
    }
}

class EmailNotificationService : INotificationService { /* ... */ }
```

```
class SmsNotificationService   : INotificationService { /* ... */ }
class PushNotificationService  : INotificationService { /* ... */ }
```

Связь с предыдущим семинаром. Принцип DIP — это теоретическое обоснование того, что делалось на семинаре по внедрению зависимостей. Тот семинар показывал, каким способом передать зависимость (через конструктор, через свойство, через метод, через контейнер). Принцип DIP объясняет, зачем это нужно: чтобы бизнес-логика не зависела от деталей инфраструктуры.

4.6 Сводная таблица

Принцип	Ключевой вопрос	Типичный признак нарушения
SRP	Одна причина для изменения?	Класс меняется по нескольким причинам
OCP	Расширение без модификации?	Новый тип требует правки старого кода
LSP	Подкласс замещает базовый?	`NotImplementedException` или сюрпризы в наследнике
ISP	Клиент не зависит от лишнего?	«Толстые» интерфейсы; `NotSupportedException`
DIP	Зависим от абстракций?	Оператор new для инфраструктуры внутри бизнес-логики

5 Рефакторинг

5.1 Определение

Мартин Фаулер в книге «Рефакторинг» формулирует: рефакторинг — это изменение внутренней структуры программного кода, не меняющее его внешнего поведения. Ключевое слово здесь — «не меняющее»: после рефакторинга программа делает ровно то же, что и до него, но делает это более понятным образом.

Важное следствие: гарантом неизменности поведения выступают тесты. Без модульных тестов рефакторинг превращается в гадание: мы правим код и надеемся, что ничего не сломали. С тестами — правим код и проверяем; при поломке тестов возвращаемся на шаг назад. Именно

поэтому тема рефакторинга тесно связана с темой модульного тестирования.

5.2 Приёмы рефакторинга на одном примере

Разберём пошаговый рефакторинг одного класса. Это позволит увидеть, как разные приёмы складываются в общий процесс.

Исходная точка — метод, который фильтрует сотрудников, печатает сводку и сохраняет отчёт в CSV:

```
class EmployeeProcessor
{
    public void Do(List<Employee> e, string d, decimal m)
    {
        List<Employee> r = new List<Employee>();
        for (int i = 0; i < e.Count; i++)
        {
            if (e[i].Department == d)
            {
                if (e[i].Salary >= m)
                {
                    if (!string.IsNullOrEmpty(e[i].Name))
                    {
                        r.Add(e[i]);
                    }
                }
            }
        }

        decimal t = 0;
        for (int j = 0; j < r.Count; j++)
        {
            t += r[j].Salary;
            Console.WriteLine("Name: " + r[j].Name + " Dept: " +
                              r[j].Department + " Salary: " + r[j].Salary)
;
        }

        decimal avg = r.Count > 0 ? t / r.Count : 0;
        Console.WriteLine("Total: " + t + " Count: " + r.Count + " Avg: "
+ avg);

        string content = "";
        for (int k = 0; k < r.Count; k++)
            content += r[k].Name + "," + r[k].Department + "," + r[k].Sala
ry + "\n";
        File.WriteAllText("report_" + d + "_" + DateTime.Now.Ticks + ".csv
", content);
    }
}
```

Применим к нему пять приёмов рефакторинга последовательно.

Приём 1. Переименование (Rename). В Visual Studio — F2. Заменяем однобуквенные имена:

```
public void Do(List<Employee> employees, string department, decimal minimu
mSalary)
{
    List<Employee> filteredEmployees = new List<Employee>();
    for (int i = 0; i < employees.Count; i++)
    {
        if (employees[i].Department == department &&
            employees[i].Salary >= minimumSalary &&
            !string.IsNullOrEmpty(employees[i].Name))
        {
            filteredEmployees.Add(employees[i]);
        }
    }
    // ...
}
```

Приём 2. Извлечение метода (Extract Method). В Visual Studio — выделить код и выбрать Ctrl+. → «Extract Method». Разделяем метод по задачам:

```
public void ProcessDepartmentEmployees(
    List<Employee> employees, string department, decimal minimumSalary)
{
    List<Employee> filtered = FilterEmployees(employees, department, minim
umSalary);
    PrintEmployeeSummary(filtered);
    ExportToCsv(filtered, department);
}

private List<Employee> FilterEmployees(
    List<Employee> employees, string department, decimal minimumSalary)
{
    return employees
        .Where(e => e.Department == department
                && e.Salary     >= minimumSalary
                && !string.IsNullOrEmpty(e.Name))
        .ToList();
}

private void PrintEmployeeSummary(List<Employee> employees)
{
    foreach (var employee in employees)
        Console.WriteLine($"{employee.Name} | {employee.Department} | {emp
loyee.Salary:C}");

    decimal total   = employees.Sum(e => e.Salary);
    decimal average = employees.Count > 0 ? total / employees.Count : 0m;
```

```
    Console.WriteLine($"Итого: {employees.Count} сотрудников, " +
                      $"фонд: {total:C}, средняя: {average:C}");
}

private void ExportToCsv(List<Employee> employees, string department)
{
    string fileName = $"report_{department}_{DateTime.Now:yyyyMMdd_HHmmss}
.csv";
    string content  = string.Join("\n",
        employees.Select(e => $"{e.Name},{e.Department},{e.Salary}"));
    File.WriteAllText(fileName, content);
}
```

Приём 3. Замена магических чисел константами:

```
// ДО
if (employee.YearsOfExperience > 5)
    bonus = employee.Salary * 0.15m;

// ПОСЛЕ
private const decimal SeniorBonusRate                = 0.15m;
private const int     SeniorExperienceThresholdYears = 5;

if (employee.YearsOfExperience > SeniorExperienceThresholdYears)
    bonus = employee.Salary * SeniorBonusRate;
```

Приём 4. Замена условной логики полиморфизмом:

```
// ДО
public void PrintItem(LibraryItem item)
{
    if      (item is Book book)         Console.WriteLine($"Книга: {book.T
itle}");
    else if (item is Magazine magazine) Console.WriteLine($"Журнал: {magaz
ine.Title}");
    else if (item is Dvd dvd)           Console.WriteLine($"DVD: {dvd.Titl
e}");
}

// ПОСЛЕ
public void PrintItem(LibraryItem item)
    => Console.WriteLine(item.GetDisplayInfo());
```

Приём 5. Внедрение зависимости. Прямое применение знаний с семинара по DI:

```
interface IReportExporter
{
    void Export(IEnumerable<Employee> employees, string reportName);
}

class CsvFileExporter : IReportExporter
{
    public void Export(IEnumerable<Employee> employees, string reportName)
    {
```

```
        string content = string.Join("\n",
            employees.Select(e => $"{e.Name},{e.Department},{e.Salary}"));
        File.WriteAllText($"{reportName}.csv", content);
    }
}

class EmployeeProcessor
{
    private readonly IReportExporter _exporter;

    public EmployeeProcessor(IReportExporter exporter)
        => _exporter = exporter;

    public void ProcessDepartmentEmployees(
        List<Employee> employees, string department, decimal minimumSalary
)
    {
        var filtered = FilterEmployees(employees, department, minimumSalar
y);
        PrintEmployeeSummary(filtered);
        _exporter.Export(filtered, $"report_{department}");
    }
}
```

5.3 Сводка изменений

До рефакторинга	После рефакторинга
Один метод Do, 35 строк	Три метода по 5–10 строк
Однобуквенные имена e, d, r	Осмысленные имена
Магические числа 0,15, 5	Именованные константы
Цепочки `if/is`	Виртуальный метод
`File.WriteAllText` внутри класса	Зависимость через конструктор

5.4 Инструменты Visual Studio

Действие	Команда
Переименование	F2
Извлечение метода	Ctrl+. → Extract Method
Перемещение типа в отдельный файл	Ctrl+. → Move type to file
Извлечение интерфейса из класса	Ctrl+. → Extract interface
Все использования символа	Shift+F12
Переход к определению	F12

6 Синтез паттерна «Стратегия» из принципов SOLID

Принципы SOLID — это не пять разрозненных правил, которые заучивают по отдельности. Это связная система, из которой при последовательном применении к реальной задаче сами собой появляются проверенные временем решения, которые называют паттернами проектирования.

В этом разделе мы рассмотрим один сценарий и решим его последовательным применением каждого принципа. Получившаяся в итоге конструкция окажется точным описанием паттерна «Стратегия». Цель — не запомнить паттерн, а научиться рассуждать так, чтобы паттерны появлялись естественным образом.

6.1 Исходная задача

Библиотечная система начисляет штрафы за просрочку возврата книг. Правила разные для разных категорий читателей:

- обычный читатель — 5 рублей в день;
- студент — 2 рубля в день (льгота);
- пенсионер — первые семь дней бесплатно, далее по 3 рубля в день;
- сотрудник библиотеки — штраф не начисляется.

Первая версия реализации:

```
class FineCalculator
{
    public decimal Calculate(string readerCategory, int daysOverdue)
    {
        if (readerCategory == "Regular")
        {
            return daysOverdue * 5m;
        }
        else if (readerCategory == "Student")
        {
            return daysOverdue * 2m;
        }
        else if (readerCategory == "Senior")
        {
            return daysOverdue > 7 ? (daysOverdue - 7) * 3m : 0m;
        }
```

```
        else if (readerCategory == "Staff")
        {
            return 0m;
        }
        else
        {
            throw new ArgumentException("Неизвестная категория читателя");
        }
    }
}
```

Код работает. Но у него есть недостатки, которые мы будем последовательно устранять.

6.2 Шаг 1. Применяем OCP

Добавление новой категории читателей (например, «школьник») потребует редактирования метода Calculate. Это нарушение OCP: класс не закрыт для модификации.

Принцип подсказывает решение: каждое правило расчёта должно существовать в виде отдельного класса, и класс FineCalculator должен работать с ними через общий абстрактный тип. Введём абстрактный класс правила:

```
abstract class FineRule
{
    public abstract decimal Calculate(int daysOverdue);
}

class RegularFineRule : FineRule
{
    public override decimal Calculate(int daysOverdue)
        => daysOverdue * 5m;
}

class StudentFineRule : FineRule
{
    public override decimal Calculate(int daysOverdue)
        => daysOverdue * 2m;
}

class SeniorFineRule : FineRule
{
    private const int GraceDays = 7;
    public override decimal Calculate(int daysOverdue)
        => daysOverdue > GraceDays ? (daysOverdue - GraceDays) * 3m : 0m;
}
```

```
class StaffFineRule : FineRule
{
    public override decimal Calculate(int daysOverdue) => 0m;
}
```

Теперь добавление новой категории не требует изменения существующих классов — достаточно создать ещё одного наследника FineRule.

6.3 Шаг 2. Проверяем SRP

У класса FineCalculator сейчас две ответственности: он и выбирает правило по категории, и применяет его. Разделим эти ответственности. Выбор правила — не задача калькулятора; калькулятору достаточно получить готовое правило и использовать его.

```
class FineCalculator
{
    private readonly FineRule _rule;

    public FineCalculator(FineRule rule)
    {
        _rule = rule;
    }

    public decimal Calculate(int daysOverdue)
        => _rule.Calculate(daysOverdue);
}
```

Теперь FineCalculator делает одно: применяет переданное ему правило. Вопрос о том, какое именно правило выбрать для конкретного читателя, решается снаружи — в точке сборки.

6.4 Шаг 3. Проверяем DIP

Класс FineCalculator зависит от абстрактного класса FineRule, а не от конкретных правил. Конкретные классы правил передаются через конструктор — это внедрение зависимости, изученное на предыдущем семинаре. Принцип DIP соблюдён: бизнес-логика (калькулятор) зависит от абстракции (правила), а не от деталей (конкретных формул).

6.5 Шаг 4. Проверяем LSP

Убедимся, что любой наследник FineRule корректно замещает базовый класс. В нашем случае это так: все наследники возвращают неотрицательное число денег за неотрицательное число дней, ни один не бросает исключений, ни один не имеет скрытых побочных эффектов. Контракт базового класса соблюдается.

6.6 Шаг 5. Проверяем ISP

Интерфейс (в нашем случае — абстрактный класс) содержит ровно один метод — Calculate. Клиент использует этот метод. Лишнего в контракте нет. ISP соблюдён.

6.7 Что получилось

Последовательно применив принципы SOLID к задаче, мы пришли к следующей конструкции:

- есть абстракция алгоритма — FineRule;
- есть набор конкретных реализаций алгоритма — RegularFineRule, StudentFineRule и так далее;
- есть контекст, который выбирает и применяет одну из реализаций, не зная об их различиях — FineCalculator.

Эта конструкция имеет собственное имя — паттерн «Стратегия» (Strategy). Она описана в классической книге «Банды четырёх» («Приёмы объектно-ориентированного проектирования. Паттерны проектирования», 1994) и применяется в миллионах программ по всему миру. Мы не заучивали паттерн — мы вывели его заново, следуя принципам.

6.8 Использование в точке сборки

Осталось собрать систему. В полном соответствии с материалом семинара по DI, выбор конкретного правила выполняется в точке сборки:

```
// Вспомогательный метод — сопоставление категории и правила
static FineRule GetFineRuleFor(string readerCategory) => readerCategory sw
itch
{
    "Regular" => new RegularFineRule(),
    "Student" => new StudentFineRule(),
    "Senior"  => new SeniorFineRule(),
    "Staff"   => new StaffFineRule(),
    _         => throw new ArgumentException("Неизвестная категория читате
ля")
};

// Пример использования
FineRule       rule       = GetFineRuleFor("Senior");
FineCalculator calculator = new FineCalculator(rule);
decimal        fine       = calculator.Calculate(daysOverdue: 10);
// Результат: (10 - 7) * 3 = 9 рублей
```

Обратите внимание: ветвление по категории читателя никуда не исчезло — оно сосредоточилось в одном-единственном месте, в точке сборки. Это естественно: где-то ведь должно быть принято решение, какое правило использовать. Но теперь это решение не размазано по бизнес-логике, а изолировано от неё.

6.9 Выводы по разделу

Паттерны проектирования — не произвольные рецепты, которые нужно заучивать. Это решения, которые закономерно получаются при последовательном применении принципов SOLID к типовым задачам. Если принципы усвоены, большинство типовых паттернов выводятся из них самостоятельно; остаётся лишь узнать их устоявшиеся имена.

Найдя код, который нарушает какой-либо принцип, нужно применять принцип — и шаг за шагом код превращается в хорошо спроектированный.

7 Использование ИИ для рефакторинга

7.1 Практический эксперимент

Современные большие языковые модели (LLM) — ChatGPT, GitHub Copilot, Claude и другие — умеют выполнять рефакторинг достаточно качественно. Рассмотрим практический сценарий.

Возьмём «грязный» фрагмент кода из начала семинара и отправим его LLM со следующим промптом:

Выполни рефакторинг приведённого кода на языке C#. Применяй принципы SOLID и чистого кода. После каждого изменения объясни, какой принцип применён и почему. Не изменяй внешнее поведение кода.

7.2 Анализ результата

Результат обычно содержит разумные правки: осмысленные имена, выделение констант, разбиение длинного метода на короткие, применение LINQ. Однако к нему следует относиться критически.

Типичные сильные стороны ответов LLM:

- знание соглашений языка и хорошая стилистика;
- корректное применение базовых приёмов рефакторинга;
- способность объяснить каждое изменение.

Типичные слабые стороны:

- незнание контекста проекта: имена могут быть «технически правильными», но не соответствующими принятой в системе терминологии;
- склонность к оверинжинирингу — нарушению KISS и YAGNI через введение абстракций «впрок»;
- возможность изменения граничных случаев — например, обработки пустого списка или null-значений;

- формальное применение принципов без учёта того, уместны ли они в конкретной задаче.

7.3 Рекомендуемый порядок работы

Перед тем как применять предложения ИИ, целесообразно:

- написать модульные тесты на исходный код — они зафиксируют текущее поведение;
- получить предложение от LLM;
- прогнать тесты на новом варианте — убедиться, что поведение не изменилось;
- критически оценить каждое изменение: понятно ли, зачем оно сделано; соответствует ли оно реальным требованиям; не вводит ли оно преждевременную сложность;
- принять изменения целиком, частично или отклонить — решение остаётся за разработчиком.

7.4 Главный тезис

Искусственный интеллект в задаче рефакторинга — это инструмент, усиливающий опытного инженера, но не заменяющий его. Без понимания принципов SOLID, IoC/DI и т.д. невозможно оценить, правильный ли совет даёт ИИ. Программист, не знающий принципов, не может отличить удачный рефакторинг от внешне красивого, но неудачного.

8 Практическое задание

8.1 Условие

Ниже приведён класс LibraryService, в котором намеренно допущено несколько нарушений рассмотренных принципов. Задача студента —

обнаружить все нарушения, объяснить, какой принцип нарушен в каждом случае, и выполнить рефакторинг.

```
class LibraryService
{
    private List<object> _items = new List<object>();

    public void AddBook(string title, string author, int year)
    {
        if (string.IsNullOrWhiteSpace(title))
            throw new ArgumentException("Название не может быть пустым");
        if (string.IsNullOrWhiteSpace(author))
            throw new ArgumentException("Автор не может быть пустым");
        if (year < 1000 || year > DateTime.Now.Year)
            throw new ArgumentException("Некорректный год издания");

        _items.Add(new { Type = "Book", Title = title, Author = author, Ye
ar = year });

        File.AppendAllText("library.log",
            $"{DateTime.Now:u}: Добавлена книга «{title}»\n");
    }

    public void AddMagazine(string title, int issueNumber)
    {
        if (string.IsNullOrWhiteSpace(title))
            throw new ArgumentException("Название не может быть пустым");
        if (issueNumber <= 0)
            throw new ArgumentException("Номер выпуска должен быть положит
ельным");

        _items.Add(new { Type = "Magazine", Title = title, IssueNumber = i
ssueNumber });

        File.AppendAllText("library.log",
            $"{DateTime.Now:u}: Добавлен журнал «{title}»\n");
    }

    public void PrintReport()
    {
        Console.WriteLine($"=== Отчёт: {_items.Count} элементов ===");

        foreach (var item in _items)
        {
            string title = item.GetType().GetProperty("Title")?.GetValue(i
tem)?.ToString()
                           ?? "Без названия";
            bool isBook  = item.GetType().GetProperty("Author") != null;
            Console.WriteLine(isBook ? $"Книга: {title}" : $"Журнал: {titl
e}");
        }

        File.WriteAllText("report.txt",
            $"Всего элементов: {_items.Count}\nДата: {DateTime.Now:u}");
```

```
    }
}
```

8.2 Требования к выполнению

- Для каждого обнаруженного нарушения указать, какой принцип (из SOLID, DRY, KISS, YAGNI) нарушен и почему.
- Выполнить рефакторинг, устранив обнаруженные нарушения. Использовать инструменты Visual Studio.
- Привести список применённых приёмов рефакторинга.

8.3 Ожидаемые нарушения

Место	Принцип	Приём рефакторинга
Класс одновременно хранит данные, валидирует, логирует и формирует отчёт	SRP	Выделение классов `LibraryItemValidator`, `LibraryReportPrinter`
Валидация названия дублируется в двух методах	DRY	Extract Method → `ValidateTitle()`
`List<object>` и использование рефлексии вместо иерархии классов	OCP	Введение абстрактного класса `LibraryItem` с виртуальным `GetDisplayInfo()`
`File.AppendAllText` прямо в методах добавления	DIP	Внедрение `ILogger` через конструктор
`File.WriteAllText` внутри метода `PrintReport`	SRP	Выделение отдельной ответственности по экспорту отчёта

9 Выводы

- Чистый код — это свойство кода, делающее его понятным для человека. В долгосрочной перспективе читаемость кода важнее скорости его написания, поскольку код читается многократно.
- Принципы DRY, KISS и YAGNI — три близких по духу правила борьбы с ненужной сложностью. DRY требует устранения дублирования знания, KISS — отказа от излишней сложности в решении, YAGNI — отказа от реализации функциональности «впрок». Принципы иногда противоречат друг другу, и опытный разработчик ищет разумный компромисс.
- Принципы SOLID — пять взаимосвязанных правил, которые вместе описывают облик хорошо спроектированного класса. Они применимы не отдельно, а как система: нарушение одного нередко влечёт за собой нарушение остальных, и наоборот, устранение одного нарушения часто помогает устранить соседние.
- Рефакторинг — это навык постоянного улучшения структуры кода без изменения его поведения. Основные приёмы (Rename, Extract Method, Replace Magic Number with Constant, Replace Conditional with Polymorphism, Introduce Dependency) поддержаны инструментами Visual Studio. Гарантом корректности рефакторинга выступают модульные тесты.
- Паттерны проектирования — закономерный результат последовательного применения принципов SOLID к типовым задачам. Умение выводить паттерн из принципов важнее, чем способность процитировать его определение.
- Искусственный интеллект — это инструмент, усиливающий знающего разработчика и почти бесполезный для незнающего. Понимание принципов остаётся необходимым условием осмысленной работы с любыми автоматическими средствами.